```
CREATE SCHEMA IF NOT EXISTS colegio_san_agustin;
SET search_path TO colegio_san_agustin;

-- Tabla 1: DOCENTE
CREATE TABLE DOCENTE (
  id_docente SERIAL PRIMARY KEY,
  documento_identidad VARCHAR(20) UNIQUE NOT NULL,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  especialidad VARCHAR(50) NOT NULL,
  telefono VARCHAR(15),
  email_institucional VARCHAR(100) UNIQUE NOT NULL,
  activo BOOLEAN DEFAULT TRUE
);

-- Tabla 2: ESTUDIANTE
CREATE TABLE ESTUDIANTE (
  id_estudiante SERIAL PRIMARY KEY,
  documento_identidad VARCHAR(20) UNIQUE NOT NULL,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  email_acudiente VARCHAR(100),
  telefono_acudiente VARCHAR(15),
  estado_academico VARCHAR(20) CHECK (estado_academico IN ('Activo', 'Retirado',
'Graduado')) NOT NULL
```

```
);

-- Tabla 3: ASIGNATURA
CREATE TABLE ASIGNATURA (
   id_asignatura SERIAL PRIMARY KEY,
   codigo_materia VARCHAR(20) UNIQUE NOT NULL,
   nombre VARCHAR(100) NOT NULL,
   grado VARCHAR(5) CHECK (grado IN ('6°', '7°', '8°', '9°', '10°', '11°')) NOT NULL,
   intensidad_horaria INT CHECK (intensidad_horaria > 0),
   descripcion TEXT
);

-- Tabla 4: CURSO_GRUPO
CREATE TABLE CURSO_GRUPO (
   id_curso SERIAL PRIMARY KEY,
   id_asignatura INT NOT NULL REFERENCES ASIGNATURA(id_asignatura),
   id_docente INT NOT NULL REFERENCES DOCENTE(id_docente),
   periodo_lectivo VARCHAR(20) NOT NULL,
   grupo VARCHAR(2) CHECK (grupo IN ('A', 'B', 'C')) NOT NULL,
   cupo_maximo INT NOT NULL CHECK (cupo_maximo > 0),
   cupo_disponible INT NOT NULL CHECK (cupo_disponible >= 0),
   aula_asignada VARCHAR(20)
);

-- Tabla 5: MATRICULA
CREATE TABLE MATRICULA (
```

```
  id_matricula SERIAL PRIMARY KEY,
  id_estudiante INT NOT NULL REFERENCES ESTUDIANTE(id_estudiante),
  id_curso INT NOT NULL REFERENCES CURSO_GRUPO(id_curso),
  fecha_matricula DATE DEFAULT CURRENT_DATE,
  estado VARCHAR(20) CHECK (estado IN ('Inscrita', 'Anulada', 'Finalizada')) NOT NULL,
  promedio_final NUMERIC(3,2) DEFAULT 0.00
);

-- Tabla 6: CALIFICACION
CREATE TABLE CALIFICACION (
  id_calificacion SERIAL PRIMARY KEY,
  id_matricula INT NOT NULL REFERENCES MATRICULA(id_matricula),
  periodo_corte INT CHECK (periodo_corte IN (1, 2, 3, 4)) NOT NULL,
  porcentaje_peso NUMERIC(5,2) CHECK (porcentaje_peso BETWEEN 1 AND 100) NOT NULL,
  nota_obtenida NUMERIC(3,2) CHECK (nota_obtenida BETWEEN 0.00 AND 5.00) NOT NULL,
  observaciones TEXT
);
```

TR_1:

```
CREATE OR REPLACE FUNCTION fn_validar_descontar_cupo() RETURNS TRIGGER AS $$
DECLARE
  v_cupo_disponible INT;
  v_estado_estudiante VARCHAR;
BEGIN
```

```
  SELECT estado_academico INTO v_estado_estudiante FROM ESTUDIANTE WHERE
id_estudiante = NEW.id_estudiante;

  IF v_estad    o_estudiante != 'Activo' THEN

    RAISE EXCEPTION 'El estudiante no está Activo [Estado Actual: %]',
v_estado_estudiante;

  END IF;


  SELECT cupo_disponible INTO v_cupo_disponible FROM CURSO_GRUPO WHERE
id_curso = NEW.id_curso;

  IF v_cupo_disponible <= 0 THEN

    RAISE EXCEPTION 'No hay cupos disponibles en el curso %.', NEW.id_curso;

  END IF;


  UPDATE CURSO_GRUPO SET cupo_disponible = cupo_disponible - 1 WHERE
id_curso = NEW.id_curso;

  RETURN NEW;

END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER TR_ValidarYDescontarCupo

BEFORE INSERT ON MATRICULA

FOR EACH ROW EXECUTE FUNCTION fn_validar_descontar_cupo();


TR_2:

CREATE OR REPLACE FUNCTION fn_restaurar_cupo() RETURNS TRIGGER AS $$

BEGIN

  IF OLD.estado = 'Inscrita' AND NEW.estado = 'Anulada' THEN
```

```
      UPDATE CURSO_GRUPO SET cupo_disponible = cupo_disponible + 1 WHERE
id_curso = NEW.id_curso;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TR_RestaurarCupoPorAnulacion
AFTER UPDATE ON MATRICULA
FOR EACH ROW EXECUTE FUNCTION fn_restaurar_cupo();

TR_3:
CREATE OR REPLACE FUNCTION fn_calcular_promedio() RETURNS TRIGGER AS $$
DECLARE
   v_promedio NUMERIC(3,2);
BEGIN
   SELECT COALESCE(SUM(nota_obtenida * (porcentaje_peso / 100.0)), 0) INTO
v_promedio
   FROM CALIFICACION WHERE id_matricula = NEW.id_matricula;

   UPDATE MATRICULA SET promedio_final = v_promedio WHERE id_matricula =
NEW.id_matricula;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER TR_CalcularPromedioFinal
```

```
AFTER INSERT OR UPDATE ON CALIFICACION

FOR EACH ROW EXECUTE FUNCTION fn_calcular_promedio();
```

Vista 1: Cursos Disponibles y Cupos

```
CREATE OR REPLACE VIEW V_CursosDisponiblesYCupo AS

SELECT

   a.codigo_materia, a.nombre AS asignatura, a.grado,

   cg.grupo, d.nombres || ' ' || d.apellidos AS docente,

   cg.cupo_maximo, (cg.cupo_maximo - cg.cupo_disponible) AS inscritos_totales,
cg.cupo_disponible

FROM CURSO_GRUPO cg

JOIN ASIGNATURA a ON cg.id_asignatura = a.id_asignatura

JOIN DOCENTE d ON cg.id_docente = d.id_docente

WHERE cg.cupo_disponible > 0;
```

Vista 2: Boletín

```
CREATE OR REPLACE VIEW V_BoletinEstudiantilConsolidado AS

SELECT

   e.nombres || ' ' || e.apellidos AS estudiante,

   a.nombre AS asignatura,

   cg.periodo_lectivo,

   STRING_AGG('Corte ' || c.periodo_corte || ': ' || c.nota_obtenida, ', ' ORDER BY
c.periodo_corte) AS calificaciones_por_corte,

   m.promedio_final AS promedio_acumulado,

   CASE WHEN m.promedio_final >= 3.0 THEN 'Aprobado' ELSE 'Reprobado' END AS
estado_aprobatorio

FROM MATRICULA m
```

```
JOIN ESTUDIANTE e ON m.id_estudiante = e.id_estudiante
JOIN CURSO_GRUPO cg ON m.id_curso = cg.id_curso
JOIN ASIGNATURA a ON cg.id_asignatura = a.id_asignatura
LEFT JOIN CALIFICACION c ON m.id_matricula = c.id_matricula
GROUP BY e.nombres, e.apellidos, a.nombre, cg.periodo_lectivo, m.promedio_final;
```

Vista 3: Carga y Rendimiento Docente

```
CREATE OR REPLACE VIEW V_CargaDocenteYRendimiento AS
SELECT
  d.nombres || ' ' || d.apellidos AS docente,
  cg.periodo_lectivo,
  COUNT(DISTINCT cg.id_curso) AS numero_de_cursos,
  COUNT(m.id_matricula) AS total_estudiantes,
  COALESCE(ROUND(AVG(m.promedio_final), 2), 0) AS promedio_general,
  COALESCE(ROUND((SUM(CASE WHEN m.promedio_final >= 3.0 THEN 1 ELSE 0
END)::NUMERIC / NULLIF(COUNT(m.id_matricula), 0)) * 100, 2), 0) AS
tasa_aprobacion_porcentaje
FROM DOCENTE d
JOIN CURSO_GRUPO cg ON d.id_docente = cg.id_docente
LEFT JOIN MATRICULA m ON cg.id_curso = m.id_curso AND m.estado != 'Anulada'
GROUP BY d.id_docente, cg.periodo_lectivo;
```

```
INSERT INTO DOCENTE (documento_identidad, nombres, apellidos, especialidad,
telefono, email_institucional, activo) VALUES
('1001', 'Carlos', 'Gómez', 'Matemáticas', '300111', 'carlos@agustin.edu', true),
('1002', 'Lucía', 'Martínez', 'Ciencias', '300112', 'lucia@agustin.edu', true),
```

('1003', 'Miguel', 'Pérez', 'Humanidades', '300113', 'miguel@agustin.edu', true),

('1004', 'Ana', 'Rojas', 'Física', '300114', 'ana@agustin.edu', true),

('1005', 'Juan', 'Díaz', 'Química', '300115', 'juan@agustin.edu', true),

('1006', 'María', 'López', 'Deportes', '300116', 'maria@agustin.edu', true),

('1007', 'David', 'Suárez', 'Artes', '300117', 'david@agustin.edu', true),

('1008', 'Laura', 'García', 'Inglés', '300118', 'laura@agustin.edu', true),

('1009', 'Jorge', 'Castro', 'Tecnología', '300119', 'jorge@agustin.edu', true),

('1010', 'Diana', 'Vega', 'Filosofía', '300120', 'diana@agustin.edu', false);

INSERT INTO ESTUDIANTE (documento_identidad, nombres, apellidos, fecha_nacimiento, email_acudiente, telefono_acudiente, estado_academico) VALUES

('2001', 'Andrés', 'Muzu', '2004-10-14', 'papa_andres@mail.com', '3102221', 'Activo'),

('2002', 'Sofia', 'Ruiz', '2005-02-10', 'mama_sofia@mail.com', '3102222', 'Activo'),

('2003', 'Felipe', 'Luna', '2004-05-20', 'papa_felipe@mail.com', '3102223', 'Activo'),

('2004', 'Camila', 'Cruz', '2006-08-15', 'mama_camila@mail.com', '3102224', 'Activo'),

('2005', 'Daniel', 'Mora', '2005-11-30', 'papa_daniel@mail.com', '3102225', 'Retirado'),

('2006', 'Valeria', 'Páez', '2004-12-05', 'mama_valeria@mail.com', '3102226', 'Activo'),

('2007', 'Esteban', 'Ríos', '2003-03-25', 'papa_esteban@mail.com', '3102227', 'Graduado'),

('2008', 'Paula', 'Gil', '2006-01-18', 'mama_paula@mail.com', '3102228', 'Activo'),

('2009', 'Mateo', 'Sol', '2005-07-22', 'papa_mateo@mail.com', '3102229', 'Activo'),

('2010', 'Isabella', 'Soto', '2006-09-10', 'mama_isabella@mail.com', '3102230', 'Activo');

INSERT INTO ASIGNATURA (codigo_materia, nombre, grado, intensidad_horaria, descripcion) VALUES

('MAT6', 'Matemáticas Básicas', '6°', 5, 'Introducción al álgebra'),

('CIE6', 'Ciencias Naturales', '6°', 4, 'Biología celular básica'),

('HUM7', 'Historia', '7°', 3, 'Historia medieval'),

('FIS10', 'Física', '10°', 4, 'Mecánica clásica'),

('QUI10', 'Química', '10°', 4, 'Química inorgánica'),

('ING11', 'Inglés Avanzado', '11°', 5, 'Preparación pruebas saber'),

('TEC9', 'Sistemas', '9°', 2, 'Lógica y algoritmos'),

('ART8', 'Artes Plásticas', '8°', 2, 'Técnicas de pintura'),

('DEP6', 'Educación Física', '6°', 2, 'Atletismo'),

('FIL11', 'Filosofía', '11°', 3, 'Filosofía moderna');

INSERT INTO CURSO_GRUPO (id_asignatura, id_docente, periodo_lectivo, grupo, cupo_maximo, cupo_disponible, aula_asignada) VALUES

(1, 1, '2026-II', 'A', 30, 30, 'Aula 101'),

(2, 2, '2026-II', 'A', 25, 25, 'Lab 1'),

(3, 3, '2026-II', 'B', 30, 30, 'Aula 102'),

(4, 4, '2026-II', 'A', 20, 20, 'Lab 2'),

(5, 5, '2026-II', 'A', 20, 20, 'Lab 3'),

(6, 8, '2026-II', 'C', 35, 35, 'Audiovisuales'),

(7, 9, '2026-II', 'A', 20, 20, 'Sala Info'),

(8, 7, '2026-II', 'B', 25, 25, 'Taller 1'),

(9, 6, '2026-II', 'A', 40, 40, 'Cancha'),

(10, 10, '2026-I', 'A', 30, 30, 'Aula 103');

-- El Trigger restará los cupos automáticamente aquí

INSERT INTO MATRICULA (id_estudiante, id_curso, estado) VALUES

(1, 1, 'Inscrita'),

(2, 1, 'Inscrita'),

(3, 2, 'Inscrita'),

(4, 3, 'Inscrita'),

(6, 4, 'Inscrita'),

(8, 5, 'Inscrita'),

(9, 6, 'Inscrita'),

(10, 7, 'Inscrita'),

(1, 8, 'Inscrita'),

(2, 9, 'Inscrita');

-- El Trigger calculará el promedio en la tabla Matrícula automáticamente aquí

INSERT INTO CALIFICACION (id_matricula, periodo_corte, porcentaje_peso, nota_obtenida, observaciones) VALUES

(1, 1, 25.0, 4.0, 'Buen inicio'),

(1, 2, 25.0, 4.5, 'Mejora continua'),

(2, 1, 30.0, 3.0, 'Debe participar más'),

(3, 1, 50.0, 5.0, 'Excelente'),

(4, 1, 25.0, 2.0, 'Faltó a clase'),

(5, 1, 25.0, 4.2, 'Buen trabajo'),

(6, 1, 25.0, 3.5, 'Regular'),

(7, 1, 20.0, 4.8, 'Destacado'),

(8, 1, 30.0, 3.9, 'Bien'),

(9, 1, 50.0, 4.1, 'Creativo');